# Dmitry Bagaev: "RxInfer.jl updates and development"

*54:06*

hello we're back and with Dimitri beov
and the RX and fur developers in
community so thank you for letting us
peek and join into your meeting really
appreciative for all the collaboration
and learning that we've had with RX
Ander this year and looking forward to
your
updates uh thanks for Hing us yes so
yeah usually we have update meetings
about the development for R and fur but
since we have now active INF syposium
and we also have a new PhD students in
our lab I decided to make it a bit
special and today we so brief I will
briefly introduce arer again for people
who are not familiar maybe it and then
we will discuss actual recent
developments of it and we it's just uh
basically an open discussion right you
can you can stop me at any moment with
some questions I for don't see YouTube
so if there are some questions on
YouTube just stop me and and I'll
discuss um so okay so arxon fur uh is a
library for scalable probabilistic
program right uh and but what what is a
probabilistic programming by
itself uh and let's let's imine we want
to build an age like a drone right and
we want to build a program for it
and write some code and usually what we
do is we take some data and uh we fit it
into our algorithm and we get the result
for example we find the P right but in

reality it's much more complicated
because we don't really have precise
data precise measurements of our sensors
or maybe we have some wind uh outside so
our formula is not really correct in
this conditions and
basically we don't have exact values but
we have some uncertainty around our
parameters and how do we actually
compute the result given the uncertainty
around the parameters that's the first
question uh but more interesting
question is actually uh the backward
question so given the
observations of our path for for our
agent what were the conditions under
which it has been operated right and
usually it's also so it's cing around
some
uncertainties uh and in the in the
context of active inference uh we can go
even uh further so we can say Okay given
the desired observations like given what
we want to
see uh what actions we actually should
take given that we we don't also know
the the conditions under which we
operate so this is what essentially
probabilistic programming is
about uh and it's it's very challenging
but good news is that we have the
solution for it we we already have it we
we we know how to solve it uh and this
is this is the formula that solves
everything essentially and this is the
base
rule uh and it looks pretty simple right
it's just like one multiplication

one uh not a big deal but
it turns out that uh so this formula is
is really really challenging it's
extremely challenging to apply real
situation uh and uh why is
that well okay we we can I can I can
demonstrate why so if we have a simple
program uh with not a lot of parameters
this this formula indeed is quite simple
so you can actually solve it on a piece
of paper uh and you will get the exact
result uh and it's all fine but in real
situations uh we actually have much more
parameters and this formula it explodes
pretty quickly and as soon as you add
more parameters to your model uh you
realize that it's no longer feasible to
solve it on a piece of paper uh actually
you need to write like an entire article
about how to solve it for like some
specific specific
case uh but we we we want even more
right so we want to not just have
parameters but we want to have some
constraints in our system uh like for
example physical constraints like maybe
we have an engine power constraint and
like we have some trees around in the
environment so when we deal with such a
complex system the map behind is just uh
you cannot even comprehend it at this
point right and actually in my opinion
this is a very big problem in so active
INF community in general is that math is
pretty difficult um to to read uh and to
like to double check and and to
implement right so uh usually In
Articles we have a lot of formulas and

they they work in theory but as soon as
we start implementing it uh we make
mistakes uh our programs have bucks and
then uh it just takes a lot of time to
iterate through
that uh but ideally we we don't even
want to to think about it right we we
want to design an agent and all of this
math under good is known so this
Bas uh and what we want to do is we want
to have as many components as we want we
want to press a button uh and we want to
compile with this map we wantest works
and just deploy right and it should
result in a robust systems that makes
reasonable actions under uncertainty and
unpredictable unpredict
environment and if you think that's too
much to ask I would uh actually draw an
analogy with uh just a regular
programming so we now talking about
probalistic programming uh but regular
programming also
started um or has the same path so in
the beginning uh like a bunch of high
level professionals phds they were
designing the very very first algorithms
uh and it was error prone they they
spent months designing a very simple
program spent months designing a
computer uh but nowaday uh every yeah
student or even in schools you can just
download Python and write simple
programs you don't you don't need think
think about how computers work to do you
just express your ideas in in some sort
of high level programming language uh
and compiler does the have Lifting for

you so essentially what we want to
achieve here is to have a language for
uh solving
probabilistic programming right and you
don't have to think about math you only
want to think about your application you
want to design your draw or maybe it's a
vum cleaner or or maybe something even
more
complex
uh any questions so
far maybe from
YouTube not yet thank you
though okay so uh okay so this is kind
of the but how do we uh approach it how
how can we approach it uh and in a sense
there are lot of similar
projects uh in in various programming
languages as well and they try to solve
the same problem in a slightly different
ways right so and given that even in
this slide there are like eight
Solutions but in reality there are more
why do we need a new
one why why did we decide to write AR
firm where there are so many solutions
already developing well um the idea here
is that um for um autonomous agents we
have um a pretty unique set of
requirements so if we want to operate in
real world uh we want to have a very big
program because a program of a real
world is is quite complicated has a lot
of parameters so it should
scale uh it also should be low power
because our autonomous agents they
usually operate on battery
and you don't really want to uh have an

empty battery soon right so it should it
should preserve energy in the most
efficient way uh it should it should be
able to adapt to changes in the
environment because the the real
environment is
chaotic uh it may change maybe you
didn't account for something your model
so you should be able to change the
model fly should work in real time
should be um yeah as efficient as
possible in terms of performance and it
also must be
robust uh so we we basically are not
allowed to stop our
computation it should run and should
react the
circumstances uh and this is what arir
is essentially about and what it tries
to achieve uh and in order to achieve
this uh it combines the three main
ideas um also in the L right Factor
graphs variational inference and
reactive message passing so this these
are the three key
elements uh and we basically try to
understand here if if we combine them uh
can we can we have these properties that
I
mentioned um and so to to talk a little
bit about each of them so a factor graph
is essentially a graph representation of
a problemistic
model uh and a good thing about fun
graphs is that they are explainable and
they're not a black box right uh so we
can in we we can clearly see the
structure of the model and dependencies

in different parameters or hidden States
and also
observations uh and it's also modular
vocal we can reuse certain pieces and we
can write specialized computations for
certain
um so in arer we have a special language
to write such programs so have this at
model macro that take textual
description of a veristic
model uh and basically creates a factor
graph under the so I user I should not
create those graphs uh by hand and this
is
optimate uh and we can also create
like sub module sub models and we can
reuse them in the bigger
context and as I mentioned uh we can for
example do some local optimizations in
our model because we clearly know the
structure
of um so variation inference I assume
that most of you may be familiar but for
those of you who are not that's
basically an optimization procedure that
approximate base rule so base rule in
many situations is very challenging to
solve
directly uh so operational inference is
is a very clever technique to trade off
accuracy and maybe solve the task a bit
easier at the cost of curac yeah and
also it's it's faster because uh you
don't have to solve the original problem
exactly um so the formula actually looks
a bit more complex than base rule
itself um but yeah the math is sometimes
strange and in practice it's actually

easier to solve this problem than the
original
one uh so instead of solving the Bas R
directly we approximate the result with
a simpler set of distributions from
exponential family for example which
makes it
simp and we have
also language for for constraints for
the variational inut right so sometimes
we want to make a meanfield over our
hidden States and maybe for some
specific States B the ation so a user
also can specify different constraints
for
proced um and the last thing in our
recipe is reactive message
passing which is a central part of the
the project so and uh each message
carries a small piece of information
required to solve this very complex math
behind uh the inference
procedure uh and
um essentially in traditional message
passing algorithms they require fixed
Global shule uh which has a lot of
downsides and in our recent fur we allow
messages to flow asynchronously and each
not note in the graph simply reacts on
changes uh in nebor would and computes
the result
loly uh and it's more robust because if
part of the system doesn't work we
simply stop react on it uh but uh also
it in principle supports variable update
rates for different variables for
example
um audio signals they may update at a

higher rate than
video uh do we have any questions so
far yeah I I can ask one now and you
could go for it or address it later Arun
asked what has been the biggest yeah
Arun asked what has been the biggest
challenge building RX and fur so
far uh oh uh that's a very good question
so the biggest challenge um I I may say
that
uh the as as I mentioned where the math
behind all of it is quite
complex uh but the implementation side
is also complex so in order to build
such a system you need an expert in both
you need to understand both math and a
high level uh programming to make it
efficient so the the the the combination
of both of this
elements um is I would say
unique uh and without understanding one
or another it's it's it's basically
impossible to be to build such a system
any other
questions
okay um okay
so uh basically to write the
problemistic program we need to specify
our model or a program if you would like
uh we need to specify our variational
distribution uh and we also need to
specify how do we compute
posteriors uh so we can Define mod in
our package we can Define variational
distribution and for the minimization we
use free energy minimization
rationalization just a s single call to
INF function that combines both model

and
constraints and runs the
algorithm and it supports both static
data sets but uh as I mentioned the
whole idea about F reactivity and it
also accepts reactive streams of data
sets from external
sources uh and this is basically we
believe that these three key elements uh
they do have properties which I
mentioned earlier and they do allow us
to um Implement all of
those uh and arur uh yeah is free it's
available on GitHub we have a website we
have a
documentation uh we also presented
various conferences and also on YouTube
uh um so yeah you are we we invite you
to check it out essentially and see yeah
what it can do for
you uh yeah so if we have any other
questions right now we can
uh no because we just had
questions but again yeah free free to um
stop me at any moment free so okay but
the question is does it actually work uh
so I this example of uh the
Drone and basically if you want to
implement drone inent we Define the
Dynamics of the Drone how for example
different parameters should influence
its position and velocity and speed and
then we Define a model n and ver uh and
then basically we create an environment
for it uh and see how the Drone would
infer the action given its current space
and the go right so in this experiment
uh our drone is instructed to fly to the

red dop and it actively tries to infer
uh its current position and the
trajectory that it needs to take to
reach the goal and and also on the
bottom side of the screen you see that
we change the parameters of the world
and also the parameters of the Drone uh
reactively and yeah the system still uh
operates uh and actually to honest it
was much more difficult to create the
environment for this drone than the
Drone itself so first simplified this
stas but the environment was still a
challenge but for that we also work on a
solution so which is called Aris
environments which essentially would
also hide all of this complexity of
building environments for uh different
engion and we also we have a a set of
other uh examples uh both for static
data sets or preactive data sets uh yeah
just feel free to check out our
documentation uh and yeah we have
reproduced many interesting patient
inference tasks they a good performance
and scalability so we we are obviously
not solving all problems in this world
uh but the set of um possibilities is is
really uh other
questions yeah I I'll ask a few from the
chat so Discovery block wrote for the
Drone would RX infer be able to manage
actual image input to infer the position
or velocity of the Drone instead of
assuming the observation corresponds to
real values with random
error uh okay it's a good question not
out of the box uh it depends on your

model so in principle it's supposed to
design a model that would process images
but uh we didn't try that so uh but arer
by itself is indifferent
on on the input right so the in the the
problem with uh images as input is that
they are pretty high
dimensional uh and since RX works on
distributions this this image must be
processed as a distribution so this
might be quite challenging in practice
but that's a direction to go with the
composition of these systems so the
Drone kernel could be designed with an
app model taking in beliefs about
position or about other features and
then hypothetically like an RX infer or
alternative approach to image processing
could be brought in and concatenated and
used and then the the kernel belief
would remain the same
uh exactly yes you you're
correct okay and one more question from
Frasier what is the largest remaining
theoretical difficulty with this
approach real time inference Universal
inference um okay it's a good question
so uh the limitations we we will talk
actually a little bit about it in the
current developments but I can already
hint that indeed Universal inference is
quite difficult because uh in order to
execute the inference procedure we need
to compute
messages uh and we we can we can compute
them in many
situations but in many situations it is
still um way too

challenging and we are actively working
on that uh but yeah message computation
is one of the main challenging
components
too awesome and one last question Bert
and Arun both asked have you considered
hierarchical or nested models or any of
the
renormalization type models we've been
exploring uh great question yes we did
and actually we last year we
reimplemented completely the the
language that we use for models to
support hierarchical models and this is
one of the directions that we are
approaching currently in bab as well
uh to support to support more complex
hierar models and also recent
developments in active imprints
literature
yes questions no not
on yeah these are good questions but we
can
continue uh so uh what about our current
development
um I can so I will talk about two and
then I will give a stage to vter uh so
we have an external collaboration with
active inference Institute about graph
visualization that paid the texal
description of the model and actually
display the graph so we have a data
structure for the graph but we don't
have a function that would create a nice
PNG out of it like picture uh and we
have an ongoing project to um for the
universal inference essentially for uh
for to support more

models so for uh graph visualization as
I mentioned it's an external
collaboration uh and it goes really well
so we have an open pool
request uh and for example for this
simple model for a simple Point model
point to model I can generate this nice
graph that will will show uh the
dependencies between variables in this
model uh and I can also
generate uh more complex models or
graphs for more complex
structures uh and uh if uh yeah so and
we have also Fraser and Chris joined our
meeting so if you Fraser or Chris you
want to talk uh something uh here you
can yeah you can do that basically
yeah I'll just can anyone can everyone
hear me is that uh is this coming
through yes yes
yeah uh yeah just really quickly so I'm
I'm Fraser I'm the guy in the top right
bubble there so along with Chris and
cobus EST hoisen we have a project
effectively at the Active inference
Institute as Demitri said in
collaboration with the bias lab to to
visualize these models to be able to get
some handle on on what they look like
and ultimately to be able to even debug
where certain messages are and and you
know where issues are with respect to
actual message computations um there is
a there's a page essentially at the um I
I could show where this is but um
there's a very a publicly available page
where you can kind of see our progress
track you know what's happening and even

reach out to to to join us and
collaborate what where you will so it's
very much open for um for collaborators
and and participation
absolutely Chris is there anything you'd
like to say
nope I think you covered it all thank
you excellent back to you
Demetri uh thank you uh Chris uh or or
if you don't want to add anything we can
just
continue please continue okay that's uh
great
thanks uh and thanks for your effort by
the way this is a really cool stuff so I
was playing with to prepare for the
presentation uh and it essentially just
works yeah so there are some yeah I
suggested some
improvement uh but yeah I think it's
it's all good even in the current stage
and also uh for example
other um probabilistic programming
libraries uh in Python for example they
do similar approach uh in their modeling
and also dis uh yeah how they display
models
um okay
so um so the next thing that I want to
talk about is um yeah Universal inter
essentially and we already had a
question about
it so suppose we have this model that
essentially tells us that we have
observations why that are coming from a
gou and and the mean of this gaan is
modeled as a beta distribution and by
Beta is because the variable that is

modeled as a beta distribution is
constrained to be from 0 to
one uh so basically I say here that my
observations are constrained from 0 to
one uh and I also say that the Precision
of my observations is an exponent of R
and R is also from0 to one so it's
essentially it's a pretty simple model
uh at a GL but the reality is that this
is an extremely challenging model for uh
message
passing uh because the messages that are
uh yeah that we need to compute are not
easy to
compute um so
and the problem is that that the result
those computation is essentially a say a
bad distribution
um so it has a it has a very strange
form that is not parametric or cannot be
easily described as a parametric
distribution so we can always describe
those distributions as a set of
samples but drawing a set of samples is
uh
costly uh and U it's essentially very
difficult to make it real
time so the idea here is that okay we we
want a nice distribution parametric and
it's us usually from exponential
family and the question is here okay how
how to find this best
approximation uh and essentially aricen
fur supports algorithms or
approximations that would allow you to
specify how you would find how you like
to find those approximations and one of
the recent uh algorith is um uh

projections
so this is a is a very very new feature
so what we can do is we can specify or
instruct AR
F to use the projection algorithm to
Beta or to other distributions from
exponential
family uh and it also works around the
terministic nodes or just for individual
variables uh and if we combine these
specifications so it's a little bit more
complex than the original example but
given this specification we can solve uh
inference in this
model um and the result is pretty so I
here I just display um uh yeah generated
some data set and I tried to infer the
parameters okay so as I mentioned in the
very very beginning we have a set of
observations and we want to know the
parameters with which those observations
were
generated uh and you can see that uh the
inert or projected distributions they
are extremely close to the the ones that
I use to actually generate data
set uh and it scales pretty well
comparing to other publicistic
programming
libraries um so here on the left side
bottom left side which ches sampling uh
and it takes up to three times
or yeah real execution time to do this
task and on the bottom part we have a
automatic differentiation variation
inference algorithm which is
faster but uh we basically uh because
it's a new feature we still try to

improve the performance and the
performance is not uh ideal at the
moment and we already know uh yeah how
to improve it essentially we have a PhD
student that works on this project and
it goes really well and like around the
week ago had yet a better uh performance
which is not reflected on this slide but
U since it's like just last week we
didn't have really time to investigate
properly uh any questions so
far
what stand for
n um n it's something like no urn
sampler
uh and urn is a is a yeah term that they
use to describe their
algorithm uh the yeah it stands for no
new turn
S it's it's practically as far as I
understand one of the best sampers right
now to draw from an unnormalized
distribution and those timings by the
way uh so
uh we Implement recent in jully and
those timings are from a
different Library also written in
jully so these timings are from
tuing um okay
so we have also we had already question
about General influence models and here
I will give stage to
V hello everyone my name is V I'm also a
PG student in by lab and I'm I'm going
to have a short uh discussion about
generalized inference in discrete models
so in active inference we really like to
work with pdps partially observable Mark

of decision
processes and the simple PDP or the
vanilla PP you can see on the slide so a
joint distribution Over States
observations and transition
matrices and we see um that we have
transition uh distri distributions both
for the observation model and the state
transition distribution okay you
click so both this uh this observation
distribution and this transition
distribution they're both categorical
transition distributions but they're
slightly different right because the the
observation model just depends on one
categorical distribution or categorical
variable only the state whereas the
transition model depends on both the
state and the control
right so if we draw the factor graph
representations of both these uh these
these nodes you can see that they're
they're extremely similar but they're
slightly different now if we want to go
towards this generalized inference
there's two things we can do the first
is we just implement this new node this
uh two categoricals in one categorical
out uh and we can just be done with it
and uh we we could do control in in
discrete environments but it it wouldn't
be very general and since these
transition distributions are very
similar their functional forms are very
similar the inference rules are also
very similar so what I investigating
right now is how we can generalize this
inference in these discrete models such

that we can have an arbitrary number of categoricals coming in a transition to a categorical variable coming out so this would give us a general contingency tables are General uh discrete categorical transition distributions uh in RX
f um there are some problems with this because this B Matrix um has a rank which is proportional to the amount of categorical interfaces so for the observation model we just have a transition Matrix but if we want to do discrete control then we already have a transition tensor which map States and controls to new States currently in arer we only support these rank one or rank two D distributions so this is something that we're working on right now to generalize this uh this inference so what we are currently doing is together with Rafael we are working to replace the rank one rank two Matrix DeRay implementation in ARs and F with the general arbitrary rank tensor implementation um the the working name for now is a multi-d l we're very much open to suggestions um I am currently deriving rules for this General uh transition node so T is already in this Des dissertation has the rules for the rank one and rank two uh 10 uh factors and matrices and I generalize them to arbitrary Rank tensors and then I will make this new transition node which takes an arbitrary number of interfaces so what can we do with this

with this we can do control in discrete
models so uh you can see on this slide
that we have an agent which is
navigating a maze and this is the type
of stuff that will be possible with the
because we will have categorical States
coming in with categorical controls
coming in um and a categorical State
coming out of this so we would be able
to go towards building um agents in
discrete environments with discrete
controls so we can go ahead and solve on
the PS as soon as we're done with
this uh thanks
Al uh and here it is here's the the
entire presentation that hope gives an
idea of what ARX project is about uh and
what we are currently trying to
develop uh and yeah maybe we were a bit
too fast but we have a plenty time for
plenty of time for
discussions do I have more questions
online
or racer if you want to ask your
question
directly one but he also in
the yeah just uh I was interested uh I'm
relative I'm very very interested I did
mathematics and computer science for my
actual degree very interested in the the
mathematics of the generalized inference
and and node constraints and all of that
to what to what extent does message
passing per se the the process of
message passing um to what extent does
that does that influence issues around
non-conjugate inference is it
effectively kind of only dependent on

the content of the messages or does does
the actual process of message passing
have some kind of non-trivial effect on
uh on on the resulting inference I'm
just curious about that so uh the
problem with nonon non- conjugate
inference in message
passing uh lies in the fact that in
order to compute a message we need to
compute an
integral uh and in general computing an
integral is not possible on a computer
I'm talking about indefinite integrals
that has no boundaries so for an
integral that has a boundary from Like A
to B say you inte from A to B you can
yeah indeed decompose it into regions uh
and then solve this um integral
numerically but for unbounded integrals
that take from minus infinity to
Infinity uh basically it is impossible
right because we have finite memory of
computers and you can always create a
counter example where it doesn't work uh
and it's even worse in um in
multi-dimensional spaces so soal Cur
cursive
dimensionality so and this is why people
are coming with u many clever methods to
take those integrals via sampling so
sampling is essentially an algorithm to
take an integral so with non conjugate
inference uh so let me phrase the other
way around so with conjugate inference
those integrals they can actually be
simplified to um an exact
formula so you don't really need to
solve the original integral you solve as

much simpler problem because you can
rearrange some terms and something just
canceled out and you get the exact
answer the non conjugate inference you
cannot do that in general so you need to
actually solve this integral and here is
why you need those approximation methods
like sampling or projection which
essentially uh try to solve this problem
but at the cost of accuracy because we
cannot
afford um yeah infinite memory in our
computer awesome I see so if I just just
try and so effectively it boils down to
clever ways to compute integrals in in
ways that you know it's not going to
take too long okay exactly exactly and
you you also have to remember about uh
dimensionality of your problem so uh for
example as I mentioned one of the recent
developments that we just uh included in
our X fur would allow you to project a
high dimensional gausian
distributions uh because before um so a
gaan distribution can be described with
its mean and with its ciance
Matrix so which means that for high
dimensional G and distribution The
ciance Matrix is quite large so it
grows uh and we recently added a
simplified version of a gaou that
doesn't store the entire coari Matrix
but only the diagonal of it and only the
same element on the entire
diag so the number of parameters is
essentially linear to the dimensionality
and it scales much much better and it
allows you to yet again solve bigger

problems um yeah in in general in
general this um this is a quite
difficult
problem I I'll ask a couple questions
from live chat if that's
okay okay
okay Arun asked is rx and fur being used
in production right
now uh well we we actually have been
contacted in the past yes but I'm not
I'm not aware of uh uh yeah any company
actively using it we we uh actually we
have a spin out from uh Technical
University of bank that we yeah we
founded a startup uh from bab Group
which is called lazy Dynamics which uh
the the entire idea is to commercialize
Ren F to make it a professional toolbox
with a really good support I mean it's
it's always uh going to be free for
research uh but uh yeah lazy Dynamics
will uh
create uh ice Tools around AR fur to
make process of uh creating models
easier making process of deploying those
models easier debugging models easier U
but essentially aroner is free and
perhaps someone is already using it in
production but uh we may not know
it cool great okay next question from
Dean tickles he wrote do your rules
self-organize or do they remain constant
does the rule recognize its own limits
of applicability and adjust accordingly
I'm just wondering if these rules can
morph into heris
sixs uh uh sorry you were glitching a
bit can you repeat the question yeah do

your rules self-organize or do they
remain constant does the rule recognize
its own limits of applicability and
adjust accordingly I'm just wondering if
these rules can morph into heris
sixs uh no like message buing is if uh
the questions about message passing
update rules they work
independently uh and this is also one of
the main ideas of this approach is to
not have any sort of global
variables uh and yeah the graph is
essentially each each message is unaware
of any other message or computations
such that we can run them
asynchronously uh and do not have any
sort of shling mechanism
uh that would yeah decide how the
messages should be computed or how they
should
cooperate awesome okay next question
from Bert he wrote is it possible to add
a deep learning encoder and train it
using
messages Yeah we actually have I think
we have an example in documentation
where we use a neural network uh and we
also use a normalizing flows inside the
F so yeah that's
definitely we also have an example where
we use differential
equations uh to infer parameters of
differential
equations maybe questions from students
about the
approach
H okay okay that's good
what what do you all usually do in the

meetings or like how does it go and how
are you going to be taking things in the
coming
months well we usually yeah we we
already discussed the recent
developments and already showed that
yeah we we do exactly that we go through
the current developments and we try to
yeah see what we can do next so uh maybe
here if someone in the chat I would just
post what for example they want to do
right like what kind of
applications they try to solve with
active
inference that would help us also to
change our agenda because we have some
certain things in our agenda so we want
to continue with graph
visualizations know that for example in
the current version uh we don't have
constraints visualization but
constraints are pretty
important uh for the inference so this
is the next step in our agenda to also
Implement constraints
visualization uh we also have non-
conjugate posterior computation in our
agenda have generalized inference in our
agenda but for example people can also
comment on their uses or they can also
for example open arfer GitHub repos
itory and open discussions right so
maybe not right now maybe if someone
watches this in the recording or maybe
someone who is online but still uh don't
have a concrete uh example you can
always go to GitHub uh open discussions
uh and just drop your ideas there and

you can discuss and maybe we uh
adjust uh yeah our goals of the project
together and we are also looking for
collaborators right so uh again Fraser
and grease were super helpful things uh
but if more people want collaborate it's
a it's an open project you can always
just join the organization on GitHub and
drop your ideas and
collaborate awesome also I wanted to
highlight kass's examples at learnable
Loop and also the documentation that you
all maintain
and just looking back at our meeting
notes from the beginning of the year and
uh looking at the version history of RX
and fur it's something really new it's
really been one of the highlights of the
year and seeing how it's all developing
and a coin TOS I mean never has a such a
simple seemingly simple statistical
situation been such an amazing
learning opening and it it uh it's just
just exciting to see how the examples
start to flow
in and then those are useful for
Learning and they're also useful for
training code models and the the the
Paradigm difference with RX infer which
I think you very quickly move through
with the app model declaration and
reactive message passing is is really
distinct from the script-like flow of
essentially all other active
implementations so it's just really
exciting to see and and also that this
isn't just about action perception
modeling we also see in the examples

more conventional statistical models so
that helps bring continuity between the
joint distributional modeling modeling
of active inference and all of these
more standard statistical
problems uh I also no just uh because I
stopped presenting I have the chat
YouTube and I see a question what's the
best way to get started with
aringer uh and I think the best way to
get started is to go to the dation that
is to the page is called getting
started but yeah so basically to start
with the
documentation uh that goes for the
process of installing the library
running and first
models uh and then um we have also a big
collection of
examples as I already mentioned maybe
some of the some of the applications you
can already find here uh we have we have
also examples about active inference uh
but not
only uh we so This Global parameters
loation uses neural network under the
hood so with lstm driven dynamic
something something is brok live stream
issues we need to open the issue
back but um but basically yeah this this
example uses ban neural network uh to
train parameters and then L mus it and
we have also external examples from
kobus we have also series of tutorials
uh on our on YouTube from Toco to jail
jail
uh from Theory point of view I really
like the book that is called uh model

based machine learning by John wi by
John win so this is also a very good uh
starting
point it also explains the concept of uh
yeah Factor graphs stochastic nodes
deterministic nodes the Run inference
but also it it uh it explains the
concept of uh running inference
essentially yeah I have a question
actually so the the project of um no
conjugate postor computation M S sim to
the variational inference so how is that
created so uh variational inference uh
is is a very very good question so
variational inference uh indeed reframes
the original
as an optimization problem and you can
solve it as a black box indeed so you
you parameterize your distribution you
take uh yeah gradients and you do a
simple optimization problem and actually
it does already solve many problems but
uh in in our per case we actually do not
treat models as black boxes we decompose
those computations in a series of
smaller computations
uh which in a sense helps us because we
don't need to run this big Global
optimization
procedure but uh in the result of the
result of this decomposition so this
individual computations messages they
still may be
difficult uh and this projection
algorithm is indeed uh yeah sort of a
variational inference but at a smaller
scale okay yeah but uh what what you
maybe refer I can also refer to this ad

algorithm that I also showed in my slides automatic differentiation variational inference which yeah takes the entire model as a black box just just differentiates through it and yeah it doesn't care about contribu thank you for this any last short comments from any of the RX team okay thank you again for for letting us kind of wander into your lab meeting and really looking forward to the collaborations that will continue okay yeah thank you thank you again for inviting us uh and yeah we are looking forward all right till next time bye